

# GPUMODE Lecture Study Notes: Positional Encodings and PaTH Attention

## Core Problem the Lecture Addresses

The lecture studies a central question in modern sequence modeling:

**How should a Transformer represent position, order, and state over long sequences?**

The original Transformer self-attention mechanism is permutation-invariant when no causal mask or positional signal is present. Without position information, the attention mechanism treats the input sequence as an unordered bag of tokens. For example, the two sentences

“the cat sat on the mat”

and

“the mat sat on the cat”

contain similar tokens but have different meanings because token order changes the semantic relation among words.

The lecture focuses on two major topics:

1. **Modern positional encoding**, especially rotary positional embedding, abbreviated as RoPE.
2. **PaTH attention**, a proposed attention mechanism that replaces position-only rotations with data-dependent Householder-like transformations to improve expressivity and state tracking.

The core bottleneck is that RoPE is powerful and efficient, but it has limitations:

- It struggles with **length extrapolation**: models trained at a context length such as 4096 tokens often suffer large perplexity blow-up when evaluated far beyond that length.
- It uses **commutative rotations**, which are insufficient for modeling some non-commutative state-tracking operations.

- It is mostly **position-dependent but data-independent**: the same position receives the same rotation regardless of token content.

PaTH attention is introduced as a more expressive alternative. It uses cumulative products of structured Householder-like matrices so that the attention score can represent non-commutative state transitions, such as swaps, permutations, and flip-flop memory updates.

The key mental shift is that RoPE treats position as a fixed geometric rotation, while PaTH treats sequence processing as a learned composition of transformations. RoPE measures distance; PaTH can track state.

## Required Background

### Self-Attention Without Positional Encoding

Given token embeddings  $x_1, x_2, \dots, x_n$ , a Transformer layer forms queries, keys, and values:

$$q_i = x_i W_Q, \quad k_i = x_i W_K, \quad v_i = x_i W_V.$$

The standard attention score from query position  $i$  to key position  $j$  is

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d}},$$

where  $d$  is the query and key dimension. The attention probabilities are

$$a_{ij} = \frac{\exp(s_{ij})}{\sum_{\ell \leq i} \exp(s_{i\ell})}$$

in a causal decoder-only Transformer. The output at position  $i$  is

$$o_i = \sum_{j \leq i} a_{ij} v_j.$$

If no positional encoding is added, the dot product  $q_i^\top k_j$  depends on token content but not on the order of the sequence. This is a problem for language, code, and reasoning because order determines meaning.

## Absolute Positional Encoding

The original Transformer used sinusoidal absolute positional encodings. For position  $p$  and channel index  $r$ , the encoding is

$$\text{PE}_{p,2r} = \sin\left(\frac{p}{10000^{2r/d}}\right),$$

$$\text{PE}_{p,2r+1} = \cos\left(\frac{p}{10000^{2r/d}}\right).$$

The input becomes

$$\tilde{x}_p = x_p + \text{PE}_p.$$

This gives each position a deterministic vector. However, absolute encodings have a limitation: they correlate content with absolute index. In many sequence tasks, what matters is not only that a token is at position 37, but that two tokens are separated by some relative distance  $i - j$ .

## Relative Positional Encoding

Relative positional encoding modifies the attention score so that it can depend on the relative displacement between query and key:

$$s_{ij} = q_i^\top k_j + b_{i-j},$$

or more generally,

$$s_{ij} = q_i^\top B_{i-j} k_j,$$

where  $B_{i-j}$  is a relative-position-dependent bilinear matrix.

RoPE is one of the most successful relative positional encoding schemes because it obtains relative-position behavior while applying simple absolute-position rotations to queries and keys.

## Main Concepts

### Two-Dimensional Rotation Matrix

RoPE is built from two-dimensional rotations. The rotation matrix for angle  $\theta$  is

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

For a vector  $u \in \mathbb{R}^2$ , rotating by angle  $\theta$  gives

$$u' = R(\theta)u.$$

Important properties are:

$$R(\theta)^\top R(\theta) = I,$$

$$R(\theta)^{-1} = R(\theta)^\top = R(-\theta),$$

$$R(\theta_1)R(\theta_2) = R(\theta_1 + \theta_2),$$

and

$$R(\theta)^m = R(m\theta).$$

These identities are the mathematical reason RoPE can apply absolute rotations but produce relative attention scores.

## Block-Diagonal RoPE Rotation

For a  $d$ -dimensional query or key vector, RoPE groups channels into pairs. If  $d$  is even, then there are  $d/2$  channel pairs. Each pair receives its own frequency  $\theta_r$ .

The full RoPE matrix is block diagonal:

$$R_\Theta(p) = \text{diag} \left( R(p\theta_0), R(p\theta_1), \dots, R(p\theta_{d/2-1}) \right).$$

Here:

- $p$  is the absolute position id.
- $\theta_r$  is the rotation frequency for channel pair  $r$ .
- Each  $2 \times 2$  block rotates one pair of channels.
- Different channel pairs do not interact.

The RoPE-transformed query and key are

$$\tilde{q}_i = R_\Theta(i)q_i,$$

$$\tilde{k}_j = R_\Theta(j)k_j.$$

The attention score becomes

$$\tilde{q}_i^\top \tilde{k}_j = (R_\Theta(i)q_i)^\top (R_\Theta(j)k_j).$$

Using orthogonality,

$$\tilde{q}_i^\top \tilde{k}_j = q_i^\top R_\Theta(i)^\top R_\Theta(j)k_j.$$

Since

$$R_\Theta(i)^\top = R_\Theta(-i),$$

we get

$$R_\Theta(i)^\top R_\Theta(j) = R_\Theta(j - i).$$

Therefore,

$$\tilde{q}_i^\top \tilde{k}_j = q_i^\top R_\Theta(j - i)k_j.$$

This is the central RoPE identity:

$$\boxed{\tilde{q}_i^\top \tilde{k}_j = q_i^\top R_\Theta(j - i)k_j}$$

Although RoPE applies an absolute position-dependent transform to  $q_i$  and  $k_j$ , their dot product depends only on the relative position  $j - i$ .

RoPE is absolute in implementation but relative in the attention score.

## High-Frequency and Low-Frequency RoPE Channels

RoPE assigns different frequencies to different channel pairs. These frequencies create a division of labor.

### High-Frequency Channels

High-frequency channels have large  $\theta_r$ . Their rotations change rapidly with position:

$$p \mapsto p\theta_r.$$

A small change in  $p$  can produce a large change in the rotated vector. These channels are useful for representing local position patterns such as:

- nearest-token relations,
- local syntax,
- $n$ -gram-like behavior,
- short-range ordering,
- precise relative position.

Because the dot product changes rapidly as  $i - j$  changes, high-frequency channels act like a high-resolution ruler for local order.

### Low-Frequency Channels

Low-frequency channels have small  $\theta_r$ . Their rotations change slowly with position. For many nearby positions,

$$R(p\theta_r) \approx I.$$

Thus the dot product remains dominated by content similarity:

$$q_i^\top R_\Theta(j - i)k_j \approx q_i^\top k_j.$$

Low-frequency channels are therefore more useful for semantic similarity and long-range information.

### Division of Labor

The lecture emphasizes that RoPE balances semantic and positional information:

$$\text{high-frequency channels} \Rightarrow \text{local position},$$

$$\text{low-frequency channels} \Rightarrow \text{semantic content and long-range behavior}.$$

This explains why changing RoPE frequencies carelessly can damage performance. High-frequency channels are fragile because they encode fine local resolution; low-frequency channels are more flexible but can create length extrapolation problems.

## Length Extrapolation Failure in RoPE

A common empirical observation is that a model trained at context length  $L_{\text{train}}$  can fail when evaluated at length  $L_{\text{test}} > L_{\text{train}}$ . Perplexity can increase sharply once the sequence length goes beyond the training context.

One explanation is that some low-frequency channels do not complete a full rotation cycle during training. For a channel frequency  $\theta_r$ , the maximum training angle is

$$\phi_{\text{train},r} = L_{\text{train}}\theta_r.$$

At test time, the model may see

$$\phi_{\text{test},r} = L_{\text{test}}\theta_r,$$

where

$$\phi_{\text{test},r} > \phi_{\text{train},r}.$$

If  $\phi_{\text{test},r}$  lies in an angle range never observed during training, the model faces an out-of-distribution positional signal. This is especially severe for low-frequency channels that rotate slowly and may only cover a small portion of their cycle during training.

## Positional Interpolation

A simple fix is positional interpolation. Instead of using position  $p$  directly, use a scaled position:

$$p' = \frac{p}{\alpha},$$

where

$$\alpha = \frac{L_{\text{test}}}{L_{\text{train}}}.$$

Then the effective angle becomes

$$p'\theta_r = \frac{p}{\alpha}\theta_r.$$

At  $p = L_{\text{test}}$ , the maximum angle is

$$\frac{L_{\text{test}}}{\alpha}\theta_r = L_{\text{train}}\theta_r.$$

Thus the model does not see angles beyond its training range.

However, this scaling is uniform across all channels. It compresses both low-frequency and high-frequency channels. That is problematic because high-frequency channels encode local ordering and should not be distorted.

## NTK-Aware RoPE Scaling

NTK-aware scaling uses a frequency-dependent scaling factor. The intuition is:

preserve high-frequency channels,

interpolate low-frequency channels.

High-frequency components are more fragile because neural networks can struggle to learn high-frequency functions from low-dimensional inputs. Since the input position id is one-dimensional and is embedded into a high-dimensional representation, the high-frequency channels carry difficult-to-learn fine structure. Distorting them can damage short-context accuracy.

Let  $s_r$  be the scaling factor for channel  $r$ . Then the adjusted frequency is

$$\theta'_r = s_r\theta_r.$$

A qualitative NTK-aware scheme satisfies

$$s_r \approx 1 \quad \text{for high-frequency channels,}$$

and

$$s_r \approx \frac{1}{\alpha} \quad \text{for low-frequency channels.}$$

The attention score becomes

$$q_i^\top R_{\Theta'}(j-i)k_j,$$

where

$$\Theta' = \{\theta'_0, \theta'_1, \dots, \theta'_{d/2-1}\}.$$



The high-level rule is:

Do not lower the resolution of the local-position ruler. Stretch the long-range ruler instead.

## NTK-by-Parts and YaRN

NTK-by-parts uses a piecewise or stair-like scaling function. Channels are divided into regimes:

$$s_r = \begin{cases} 1, & \text{high-frequency channels,} \\ \text{interpolated value,} & \text{middle-frequency channels,} \\ 1/\alpha, & \text{low-frequency channels.} \end{cases}$$

Thus:

- High-frequency channels extrapolate without scaling.
- Low-frequency channels are fully interpolated.
- Middle channels combine interpolation and extrapolation.

YaRN, meaning Yet another RoPE extensioN, combines NTK-by-parts with an additional temperature scaling term. At a high level, this changes the entropy of the attention distribution when context length changes.

If attention logits are scaled by a temperature  $\tau$ , then

$$a_{ij} = \frac{\exp(s_{ij}/\tau)}{\sum_{\ell \leq i} \exp(s_{i\ell}/\tau)}.$$

A larger  $\tau$  makes attention more diffuse, while a smaller  $\tau$  makes it sharper. YaRN uses temperature adjustment to stabilize attention behavior under long-context extension.

## Hardware-Level Explanation

### RoPE in GPU Kernels

RoPE is efficient because it is local to each query and key vector. It can be applied before attention score computation:

$$q_i \leftarrow R_{\Theta}(i)q_i, \quad k_j \leftarrow R_{\Theta}(j)k_j.$$

For each pair of channels  $(2r, 2r + 1)$ ,

$$\begin{bmatrix} \tilde{x}_{2r} \\ \tilde{x}_{2r+1} \end{bmatrix} = \begin{bmatrix} \cos(p\theta_r) & -\sin(p\theta_r) \\ \sin(p\theta_r) & \cos(p\theta_r) \end{bmatrix} \begin{bmatrix} x_{2r} \\ x_{2r+1} \end{bmatrix}.$$

In code, this is just a pairwise fused operation:

$$\tilde{x}_{2r} = x_{2r} \cos(p\theta_r) - x_{2r+1} \sin(p\theta_r),$$

$$\tilde{x}_{2r+1} = x_{2r} \sin(p\theta_r) + x_{2r+1} \cos(p\theta_r).$$

This has low arithmetic cost. It is often fused into attention kernels or applied during query and key projection.

## Memory Hierarchy Considerations

In a GPU implementation, the main relevant memory spaces are:

- **HBM/global memory:** stores the input tensors, query, key, value, and KV cache.
- **L2 cache:** shared across SMs and useful for reused key and value blocks.
- **Shared memory:** used by attention kernels to stage blocks of  $Q$ ,  $K$ , and  $V$ .
- **Registers:** hold per-thread fragments, partial dot products, and softmax statistics.
- **Tensor cores:** accelerate matrix multiply operations when data types and layouts are suitable.

RoPE itself is not usually the bottleneck. Attention is dominated by memory traffic and matrix multiplication:

$$QK^\top \quad \text{and} \quad AV.$$

FlashAttention improves efficiency by tiling attention and avoiding materialization of the full  $n \times n$  attention matrix.

## FlashAttention-Style Tiling

For a query block  $Q_b$  and key-value block  $(K_c, V_c)$ , attention computes

$$S_{bc} = Q_b K_c^\top,$$

$$P_{bc} = \text{softmax}(S_{bc}),$$

$$O_b \leftarrow O_b + P_{bc} V_c.$$

FlashAttention uses online softmax statistics:

$$m_i = \max_j s_{ij},$$

$$\ell_i = \sum_j \exp(s_{ij} - m_i),$$

so that blocks can be processed without storing all logits.

## Why PaTH Needs More Than RoPE

RoPE modifies  $Q$  and  $K$  independently. Once they are rotated, the attention kernel can proceed almost identically to normal attention.

PaTH is different. It uses cumulative products of data-dependent transformations. The bilinear matrix between query  $i$  and key  $j$  depends on all intermediate transformations between  $j$  and  $i$ :

$$B_{ij} = H_i H_{i-1} \cdots H_{j+1}.$$

This creates extra work because attention between two tokens is not only a function of distance. It depends on the sequence of transformations between them.

The lecture argues that PaTH can still be made hardware-aligned by using blockwise algorithms. Within each block, cumulative Householder products can be represented using compact matrix operations, and across blocks the attention computation can be adapted from FlashAttention.

## Mathematical Reasoning

### RoPE as a Bilinear Relative Encoding

RoPE attention can be written as

$$s_{ij} = q_i^\top R_\Theta(j-i) k_j.$$

This is a bilinear form with relative-position-dependent weight matrix:

$$B_{i-j} = R_\Theta(j-i).$$

Thus,

$$s_{ij} = q_i^\top B_{i-j} k_j.$$

Because  $B_{i-j}$  is built from rotations, it is norm-preserving:

$$\|R_\Theta(j-i)x\|_2 = \|x\|_2.$$

This is useful for stability but also limits expressivity.

## Commutativity of RoPE Rotations

For a single two-dimensional rotation block,

$$R(\theta_1)R(\theta_2) = R(\theta_1 + \theta_2) = R(\theta_2 + \theta_1) = R(\theta_2)R(\theta_1).$$

Thus,

$$R(\theta_1)R(\theta_2) = R(\theta_2)R(\theta_1).$$

This means rotations in the same plane commute. RoPE uses block-diagonal matrices of independent two-dimensional rotations, so the overall transformations also commute:

$$R_\Theta(a)R_\Theta(b) = R_\Theta(b)R_\Theta(a).$$

This commutativity is a key limitation when the task itself requires non-commutative state updates.

## Non-Commutative State Tracking

Consider a sequence of swaps. Let  $S_{ab}$  swap positions  $a$  and  $b$ . In general,

$$S_{ab}S_{bc} \neq S_{bc}S_{ab}.$$

For example, with elements  $A, B, C$ , first applying  $S_{ab}$  and then  $S_{bc}$  can move  $A$  to a different final position than applying  $S_{bc}$  first and then  $S_{ab}$ .

State-tracking tasks often require such order-sensitive composition. Code execution, variable reassignment, stack updates, and symbolic manipulation are examples where updates are not commutative.

RoPE has difficulty representing this kind of operation directly because its positional transformations commute.

## Complexity Classes: TC0 and NC1

The lecture uses complexity theory to explain expressive limitations.

### TC0

The class TC0 consists of problems solvable by constant-depth, polynomial-size circuits with unbounded fan-in gates, including threshold gates. A threshold gate can be thought of as a majority-vote gate.

Informally,

$$\text{TC0} = \text{constant-depth circuits with threshold gates.}$$

TC0 can represent many arithmetic-like operations, including:

- addition,
- comparison,
- some prefix operations,
- integer multiplication,
- integer division.

The lecture states that standard Transformers and state-space models can fall into this restricted expressive regime for certain theoretical analyses.

### NC1

The class NC1 consists of problems solvable by logarithmic-depth, polynomial-size circuits with bounded fan-in Boolean gates.

Informally,

$$\text{NC1} = \text{log-depth divide-and-conquer circuits.}$$

NC1 captures computations such as:

- Boolean formula evaluation,
- permutation composition,
- certain state-tracking problems,
- problems requiring hierarchical composition.

The key contrast is

$$\text{TC0} \quad \text{is shallow and counting-like,}$$

while

NC1 supports logarithmic-depth compositional computation.

## The Five-Element Permutation Problem

The lecture discusses a minimal NC1-complete problem: composition of permutations on five elements. Let there be five elements:

$$\{0, 1, 2, 3, 4\}.$$

A permutation  $\pi$  maps each element to another position. A sequence of permutations composes as

$$\Pi = \pi_T \circ \pi_{T-1} \circ \cdots \circ \pi_1.$$

The goal is to determine the final position of an element after applying all transformations.

Each permutation can be decomposed into swaps, so it is enough to study cumulative swap composition. This is a state-tracking problem because the model must maintain the current configuration of all elements.

## Householder Reflection

A Householder matrix reflects vectors across a hyperplane. Given a unit vector  $w$  satisfying

$$\|w\|_2 = 1,$$

the classical Householder reflection is

$$H = I - 2ww^\top.$$

For any vector  $x$ ,

$$Hx = x - 2w(w^\top x).$$

This subtracts twice the projection of  $x$  onto  $w$ , producing a reflection.

The Householder matrix is symmetric:

$$H^\top = H,$$

orthogonal:

$$H^\top H = I,$$

and involutive:

$$H^2 = I.$$

### Householder Reflections Can Encode Swaps

Given two orthonormal basis vectors  $e_a$  and  $e_b$ , choose

$$w = \frac{e_a - e_b}{\|e_a - e_b\|_2} = \frac{e_a - e_b}{\sqrt{2}}.$$

Then

$$H = I - 2ww^\top.$$

Applying  $H$  to  $e_a$  gives

$$He_a = e_a - 2w(w^\top e_a).$$

Since

$$w^\top e_a = \frac{1}{\sqrt{2}},$$

we get

$$He_a = e_a - 2\left(\frac{e_a - e_b}{\sqrt{2}}\right)\left(\frac{1}{\sqrt{2}}\right) = e_a - (e_a - e_b) = e_b.$$

Similarly,

$$He_b = e_a.$$

All orthogonal directions remain unchanged. Therefore a Householder reflection can implement a swap.

$$\boxed{He_a = e_b, \quad He_b = e_a}$$

This is the mathematical bridge from linear algebra to state tracking.

## Generalized Householder Transformation

PaTH uses a generalized Householder-like transform:

$$H_t = I - \beta_t w_t w_t^\top,$$

where

$$\|w_t\|_2 = 1,$$

and

$$0 \leq \beta_t \leq 2.$$

Special cases:

$$\beta_t = 0 \quad \Rightarrow \quad H_t = I,$$

which means the model skips or ignores the update.

$$\beta_t = 1 \quad \Rightarrow \quad H_t = I - w_t w_t^\top,$$

which is a projection-like operation.

$$\beta_t = 2 \quad \Rightarrow \quad H_t = I - 2w_t w_t^\top,$$

which is a classical reflection.

Thus, PaTH can learn whether a token should:

- do nothing,
- partially erase or project information,
- reflect or swap state.

## PaTH Attention Score

RoPE attention has the form

$$s_{ij}^{\text{RoPE}} = q_i^\top R_\Theta(j-i)k_j.$$

PaTH replaces the rotation matrix with a cumulative product of Householder-like transitions. For  $j < i$ , define



$$B_{ij} = H_i H_{i-1} \cdots H_{j+1}.$$

Then the PaTH attention score is

$$s_{ij}^{\text{PaTH}} = q_i^\top B_{ij} k_j.$$

Equivalently,

$$s_{ij}^{\text{PaTH}} = q_i^\top \left( \prod_{t=j+1}^i H_t \right) k_j$$

where each

$$H_t = I - \beta_t w_t w_t^\top.$$

The important difference from RoPE is that  $H_t$  is data-dependent. The vector  $w_t$  and scalar  $\beta_t$  can be produced from the hidden state at token  $t$ :

$$w_t = \frac{W_h x_t}{\|W_h x_t\|_2},$$

$$\beta_t = 2\sigma(u^\top x_t),$$

where  $\sigma$  is the sigmoid function. This gives

$$0 < \beta_t < 2.$$

## Why PaTH Is More Expressive

PaTH can represent cumulative swap composition because:

$$H_t \quad \text{can implement a swap,}$$

and

$$H_i H_{i-1} \cdots H_{j+1} \quad \text{can implement a sequence of swaps.}$$

Since composition of swaps can represent permutation composition, and five-element permutation composition is NC1-complete, PaTH gains access to computations that RoPE-style commutative rotations cannot naturally express.

The lecture summarizes the separation as:

RoPE  $\Rightarrow$  commutative, position-only rotations,

PaTH  $\Rightarrow$  non-commutative, data-dependent transformations.

## Code Walkthrough

### Minimal RoPE Implementation

```
import torch

def apply_rope(x, cos, sin):
    """
    x:      tensor of shape [batch, heads, seq, dim]
    cos:    tensor broadcastable to [batch, heads, seq, dim // 2]
    sin:    tensor broadcastable to [batch, heads, seq, dim // 2]
    """
    x_even = x[..., 0::2]
    x_odd = x[..., 1::2]

    y_even = x_even * cos - x_odd * sin
    y_odd = x_even * sin + x_odd * cos

    y = torch.empty_like(x)
    y[..., 0::2] = y_even
    y[..., 1::2] = y_odd
    return y
```

This implements the two-dimensional rotation:

$$\begin{bmatrix} y_{2r} \\ y_{2r+1} \end{bmatrix} = \begin{bmatrix} \cos(p\theta_r) & -\sin(p\theta_r) \\ \sin(p\theta_r) & \cos(p\theta_r) \end{bmatrix} \begin{bmatrix} x_{2r} \\ x_{2r+1} \end{bmatrix}.$$

Important implementation details:

- The even and odd channels form independent two-dimensional planes.
- The operation is elementwise and memory-bandwidth-light compared with attention.
- In optimized kernels, this can be fused into query and key loading.
- Since RoPE only applies to  $Q$  and  $K$ , values  $V$  do not directly receive positional rotations.

## Constructing RoPE Frequencies

```
def rope_frequencies(dim, base=10000.0, device="cuda"):
    half_dim = dim // 2
    idx = torch.arange(half_dim, device=device, dtype=torch.float32)
    theta = 1.0 / (base ** (idx / half_dim))
    return theta

def rope_cos_sin(seq_len, dim, base=10000.0, device="cuda"):
    theta = rope_frequencies(dim, base=base, device=device)
    pos = torch.arange(seq_len, device=device, dtype=torch.float32)
    angles = pos[:, None] * theta[None, :]
    return torch.cos(angles), torch.sin(angles)
```

The frequency schedule is typically geometric:

$$\theta_r = \frac{1}{\text{base}^{r/(d/2)}}.$$

Large  $\theta_r$  produces high-frequency channels. Small  $\theta_r$  produces low-frequency channels.

## NTK-by-Parts Style Frequency Scaling

```
def piecewise_rope_scaling(theta, alpha, low_idx, high_idx):
    """
    theta:    original RoPE frequencies, shape [dim // 2]
    alpha:    context extension factor
    low_idx:  channels below this index are treated as high-frequency
    high_idx: channels above this index are treated as low-frequency
    """
    n = theta.numel()
    device = theta.device
    r = torch.arange(n, device=device, dtype=torch.float32)

    scale = torch.ones_like(theta)

    # Middle channels interpolate linearly between no scaling and full scaling.
    middle = (r >= low_idx) & (r <= high_idx)
    t = (r - low_idx) / max(float(high_idx - low_idx), 1.0)
    scale_middle = (1.0 - t) * 1.0 + t * (1.0 / alpha)

    scale = torch.where(middle, scale_middle, scale)
    scale = torch.where(r > high_idx, torch.full_like(scale, 1.0 / alpha), scale)

    return theta * scale
```

This captures the lecture intuition:

$$\theta'_r = \begin{cases} \theta_r, & \text{high-frequency channels,} \\ s_r \theta_r, & \text{middle-frequency channels,} \\ \theta_r / \alpha, & \text{low-frequency channels.} \end{cases}$$

The purpose is to preserve local precision while extending long-context range.

## Minimal Householder Transform

```
import torch
import torch.nn.functional as F

def householder_apply(x, w, beta):
    """
    Apply  $H x = x - \beta w * (w^T x)$ .

    x:      tensor of shape [..., dim]
    w:      tensor of shape [..., dim], assumed L2-normalized
    beta:   tensor of shape [...] or [..., 1]
    """
    dot = (w * x).sum(dim=-1, keepdim=True)
    return x - beta.unsqueeze(-1) * w * dot

def normalize_w(raw_w, eps=1e-6):
    return raw_w / (raw_w.norm(dim=-1, keepdim=True) + eps)
```

This implements

$$H_t x = x - \beta_t w_t (w_t^\top x).$$

Hardware interpretation:

- Compute one dot product  $w_t^\top x$ .
- Scale the vector  $w_t$  by that scalar.
- Subtract the result from  $x$ .
- This is a rank-one update.

The operation costs  $O(d)$  per vector, not  $O(d^2)$ , because the matrix

$$H_t = I - \beta_t w_t w_t^\top$$

is never materialized.

## Naive PaTH Score for One Pair

```
def path_score_naive(q_i, k_j, ws, betas):
    """
    q_i: query vector at position i, shape [dim]
    k_j: key vector at position j, shape [dim]
    ws: list or tensor of w_t for t = j + 1, ..., i
    betas: list or tensor of beta_t for t = j + 1, ..., i
    """
    x = k_j
    for w_t, beta_t in zip(ws, betas):
        x = householder_apply(x, w_t, beta_t)
    return torch.dot(q_i, x)
```

This computes

$$q_i^\top (H_i H_{i-1} \cdots H_{j+1}) k_j.$$

This implementation is conceptually simple but inefficient for full attention because it repeats cumulative products for many  $(i, j)$  pairs. Efficient PaTH uses blockwise computation and matrix operations.

## PaTH Inference as Dynamic Key-Cache Refinement

During autoregressive decoding, when a new token arrives at time  $t$ , PaTH can update all previous keys with the new transition:

$$k_j^{(t)} = H_t k_j^{(t-1)} = k_j^{(t-1)} - \beta_t w_t \left( w_t^\top k_j^{(t-1)} \right).$$

This is a rank-one update over the key cache.

```
def update_key_cache_path(K_cache, w_t, beta_t):
    """
    K_cache: tensor of shape [num_tokens, heads, dim]
    w_t: tensor of shape [heads, dim]
    beta_t: tensor of shape [heads]
    """
    dot = (K_cache * w_t[None, :, :]).sum(dim=-1, keepdim=True)
    K_cache = K_cache - beta_t[None, :, None] * w_t[None, :, :] * dot
    return K_cache
```

After this update, decoding can use ordinary attention between the current query and the updated historical keys:

$$s_{tj} = q_t^\top k_j^{(t)}.$$

This is why the lecture says PaTH inference can be reduced to standard attention decoding after key-cache adjustment.

## Hardware Notes for the Key-Cache Update

The PaTH decoding update reads and writes the key cache:

$$K_{\text{cache}} \leftarrow H_t K_{\text{cache}}.$$

The additional cost is:

$$O(n_{\text{past}} d)$$

per new token per head. This is similar in shape to scanning the key cache for attention. The major extra memory traffic is writing the updated key cache back to HBM.

Possible optimization directions include:

- Fuse the rank-one update with attention decoding.
- Load key blocks into shared memory, update them, use them for attention, and write them back.
- Pipeline the update and attention computation across blocks.
- Skip or prune keys whose norm decays close to zero.

## Performance Intuition

### RoPE Performance

RoPE is efficient because:

- It is an elementwise pairwise transformation.
- It does not require communication across tokens.
- It can be fused with query and key loading.
- It keeps attention compatible with FlashAttention.

The cost of RoPE per token is approximately

$$O(d).$$

The cost of full attention is approximately

$$O(n^2 d)$$

for training, or

$$O(nd)$$

per decoding step with KV cache. Therefore RoPE overhead is usually small.

## Why RoPE Is Not Enough

RoPE only creates relative rotations. These rotations are:

- position-only,
- data-independent,
- norm-preserving,
- commutative within each rotation plane.

This is excellent for encoding distances, but weak for tasks requiring state changes. A variable reassignment in code, for example, is more like a state update than a distance measurement.

## PaTH Performance

PaTH is more expensive because it introduces learned cumulative transformations. However, it is designed to remain GPU-friendly:

- Each Householder transform is structured and rank-one.
- Products can be represented compactly.
- Blockwise computation can reuse FlashAttention-style tiling.
- Many operations can be expressed as matrix multiplications.
- Small block-local inverses are acceptable because block size is small.

For a block size  $B$ , some local operations may have complexity

$$O(B^3),$$

but this is over the block size, not the sequence length. Since  $B$  may be 64 or another small constant, this can be parallelized across blocks.

The expensive part of arbitrary dense transition matrices would be

$$O(d^3)$$

for matrix-matrix products, which is unacceptable. Householder structure avoids that. Applying a Householder transform costs

$$O(d),$$

and composing structured transforms can be made far cheaper than dense matrix multiplication.

## FlashAttention Adaptation for PaTH

In standard FlashAttention, one often scans key-value blocks from left to right or in an order convenient for memory. For RoPE, the scan order is flexible because the relative position can be computed from absolute positions.

For PaTH, scan order matters because the transformation is a cumulative product:

$$H_i H_{i-1} \cdots H_{j+1}.$$

The product is non-commutative, so the order cannot be rearranged arbitrarily. The PaTH FlashAttention-style kernel must process blocks in an order consistent with the transformation direction.

The lecture describes the key changes:

- Load a query block into shared memory.
- Scan key-value blocks in the required direction, typically from nearby blocks to farther blocks.
- Maintain a transformed query or transformed key representation.
- Update the query block after each key block to incorporate cumulative Householder transformations.
- Use online softmax statistics as in FlashAttention.

## Memory-Bound Versus Compute-Bound Behavior

Attention can be memory-bound or compute-bound depending on sequence length, head dimension, tiling, and hardware.

The roofline-style arithmetic intensity is

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}.$$

If

$$\text{AI} < \frac{\text{peak FLOPs}}{\text{peak memory bandwidth}},$$



the kernel is memory-bound. If

$$\text{AI} > \frac{\text{peak FLOPs}}{\text{peak memory bandwidth}},$$

the kernel is compute-bound.

PaTH adds computation, which may increase arithmetic intensity, but it also adds key-cache writes during decoding. Thus PaTH performance depends strongly on whether the extra computation can be fused and whether the additional memory traffic can be hidden.

## PaTH, Forgetting Transformers, and PaTH-FoX

### Forgetting Transformers as Data-Dependent ALiBi

Forgetting Transformers introduce a learned forget gate into attention. Let  $g_t$  be a scalar gate with

$$0 < g_t \leq 1.$$

A cumulative forget factor from position  $j$  to  $i$  is

$$G_{ij} = \prod_{t=j+1}^i g_t.$$

This can modify the attention score, for example through a log-bias:

$$s_{ij} = q_i^\top k_j + \log G_{ij}.$$

Since

$$\log G_{ij} = \sum_{t=j+1}^i \log g_t,$$

the forget gate acts as an additive attention bias.

ALiBi uses a fixed distance-dependent linear bias. Forgetting Transformers use a data-dependent forget bias. Thus, one can view Forgetting Transformers as a data-dependent version of ALiBi.

## Multiplicative Versus Additive Positional Encoding

RoPE and PaTH are multiplicative or bilinear:

$$s_{ij} = q_i^\top B_{ij} k_j.$$

Forgetting-style methods are additive:

$$s_{ij} = q_i^\top k_j + b_{ij}.$$

PaTH-FoX combines both:

$$s_{ij}^{\text{PaTH-FoX}} = q_i^\top \left( \prod_{t=j+1}^i H_t \right) k_j + \sum_{t=j+1}^i \log g_t.$$

This combines:

multiplicative state transformation

with

additive forgetting bias.

## Connection to Linear Attention and Recurrent Models

The lecture draws a connection between softmax attention, linear attention, and recurrent state-space-like models.

Softmax attention can be interpreted as a kernel method:

$$\exp(q^\top k) = \sum_{m=0}^{\infty} \frac{(q^\top k)^m}{m!}.$$

This expansion corresponds to an infinite-dimensional feature map:

$$\exp(q^\top k) = \phi(q)^\top \phi(k).$$

Linear attention uses finite-dimensional feature maps:

$$\text{sim}(q, k) = \phi(q)^\top \phi(k).$$

The lecture's conceptual point is:

softmax attention  $\Rightarrow$  large or infinite memory capacity,

while recurrent or linear-attention models often have more explicit state transitions but bounded hidden-state memory.

PaTH attempts to combine these strengths:

expressive transition dynamics + softmax memory capacity.

## Synthetic Tasks and Empirical Findings

### Multiquery S5 Swap Task

The multiquery S5 task uses five elements and a sequence of swap operations. The model receives actions such as:

$\text{swap}(a, b)$

and later receives queries such as:

where is element 4?

The model must answer the final position of the queried element. The task tests whether the model can track all five elements simultaneously.

There is no explicit chain-of-thought. The model must solve the problem in its forward pass.

The lecture reports:

RoPE fails to solve the task reliably,

while

PaTH learns the task quickly.

This supports the theory that PaTH can represent non-commutative state transitions more naturally.

## Flip-Flop Language Modeling

The flip-flop task contains actions such as:

write,      read,      ignore.

The model must output the most recently written bit when a read token occurs. This is a memory-state task. PaTH performs well because it can learn when to preserve, update, reflect, or erase state.

## A5 World Problem

The A5 world problem is related to composition in the alternating group  $A_5$ . It is another group-composition state-tracking benchmark.

The lecture reports a separation:

RoPE-like models require roughly linear depth as sequence length grows,  
whereas

PaTH can solve with logarithmic depth.

This matches the NC1 intuition: PaTH is better suited for hierarchical composition.

## Distillation from RoPE to PaTH

A practical question is how to use PaTH when most existing language models are trained with RoPE. The lecture suggests a two-stage distillation pipeline.

Let the original RoPE model be the teacher and the PaTH model be the student. The student is initialized from the teacher weights except that RoPE attention is replaced by PaTH attention.

### Stage 1: Layerwise Feature Matching

For each layer  $\ell$ , collect teacher inputs and outputs:

$$h_{\ell}^{\text{teacher,in}}, \quad h_{\ell}^{\text{teacher,out}}.$$

Train the student layer to minimize mean squared error:

$$\mathcal{L}_{\text{MSE}} = \sum_{\ell} \left\| h_{\ell}^{\text{student,out}} - h_{\ell}^{\text{teacher,out}} \right\|_2^2.$$

This reduces local mismatch caused by replacing the positional encoding mechanism.

### Stage 2: End-to-End Knowledge Distillation

Then train the full model by matching output distributions. Let

$$p_T(y \mid x)$$

be the teacher distribution and

$$p_S(y \mid x)$$

be the student distribution. Minimize

$$\mathcal{L}_{\text{KD}} = D_{\text{KL}}(p_T(\cdot \mid x) \parallel p_S(\cdot \mid x)).$$

This aligns the student with the teacher at the vocabulary-logit level.

### Continued Pretraining

The lecture also discusses continued pretraining from intermediate checkpoints. This makes architectural comparison easier because one can control:

- training data,
- number of tokens,
- optimizer schedule,
- model initialization,
- benchmark evaluation.

Reported trends suggest that PaTH improves coding and math tasks, which is consistent with the claim that these tasks require better state tracking.

### Length Extrapolation Observations

For perplexity beyond training context length:

- RoPE often blows up quickly.
- Forgetting Transformers extrapolate somewhat but can lose long-range information because they behave like a soft sliding window.
- PaTH alone improves behavior relative to RoPE but may still require long-context training.

- PaTH-FoX gives the strongest long-context signal in the lecture because it combines expressive transformations with forgetting gates.

## Common Misconceptions

### Misconception: RoPE Is an Absolute Positional Encoding

RoPE uses absolute position ids during implementation, but the attention score depends on relative position:

$$\tilde{q}_i^\top \tilde{k}_j = q_i^\top R_\Theta(j - i)k_j.$$

Thus RoPE is best understood as a relative positional encoding implemented through absolute rotations.

### Misconception: All RoPE Channels Should Be Scaled Equally for Long Context

Uniform interpolation treats all frequencies the same. But high-frequency and low-frequency channels play different roles. High-frequency channels encode local order and should often be preserved. Low-frequency channels are more suitable for interpolation.

### Misconception: Values Carry RoPE Positional Information Directly

RoPE is applied to queries and keys, not values. Positional information affects the attention weights:

$$a_{ij} = \text{softmax}_j \left( q_i^\top R_\Theta(j - i)k_j \right).$$

The output is still a weighted sum of values:

$$o_i = \sum_{j \leq i} a_{ij} v_j.$$

Thus values receive positional influence indirectly through attention weights, not through direct rotation.

### Misconception: More Expressivity Always Means Faster Models

PaTH is more expressive than RoPE for state tracking, but it is not automatically faster. It adds transition computations and may update the key cache. The

efficiency argument is that PaTH is structured enough to be implemented reasonably well, not that it is cheaper than RoPE.

## Misconception: Chain-of-Thought and Architecture Solve the Same Problem

Chain-of-thought can increase effective computational depth by generating intermediate tokens. This can help a RoPE Transformer solve state-tracking tasks. However, it increases inference cost. PaTH aims to improve the base model’s ability to perform state tracking inside a forward pass, potentially reducing the amount of generated reasoning needed.

## Practical Takeaways

1. **RoPE is a relative positional encoding in disguise.** Its core identity is

$$(R(i)q_i)^\top (R(j)k_j) = q_i^\top R(j-i)k_j.$$

2. **RoPE channels have different jobs.** High-frequency channels encode local order; low-frequency channels encode semantics and long-range position.
3. **Length extension should be frequency-aware.** Methods such as NTK-aware scaling, NTK-by-parts, and YaRN work because they avoid damaging high-frequency local structure while interpolating low-frequency channels.
4. **RoPE is limited by commutativity.** Rotations are elegant and stable, but they cannot naturally represent non-commutative state updates such as ordered swap composition.
5. **Householder transformations are useful because they are structured and expressive.** They can implement reflections, swaps, projections, and identity-like behavior.
6. **PaTH attention converts positional encoding into learned transition composition.** Its score

$$q_i^\top \left( \prod_{t=j+1}^i H_t \right) k_j$$

allows data-dependent state evolution between tokens.

7. **PaTH is motivated by state tracking.** Coding, math, symbolic reasoning, and variable updates all require maintaining and transforming state.

8. **PaTH can be made GPU-aligned but requires careful kernels.**  
Efficient implementations should use blockwise products, shared memory tiling, online softmax, and fused key-cache updates.
9. **PaTH-FoX combines two complementary mechanisms.** PaTH supplies multiplicative expressive state transitions, while forgetting gates supply additive memory control and length extrapolation behavior.

## Project or Experiment Ideas

### Experiment 1: Verify the RoPE Relative Position Identity

Implement RoPE and numerically verify that

$$(R(i)q)^\top (R(j)k) = q^\top R(j-i)k.$$

Test this for random  $q$ ,  $k$ ,  $i$ , and  $j$ .

### Experiment 2: Compare Uniform Scaling and NTK-by-Parts

Train or evaluate a small Transformer with RoPE at context length  $L_{\text{train}}$ . Then compare:

- no scaling,
- uniform positional interpolation,
- NTK-by-parts scaling,
- YaRN-style scaling.

Plot perplexity versus context length.

### Experiment 3: Synthetic Swap Tracking

Create a dataset of five-element swap sequences. Each example contains a list of swaps and a query asking for the final position of one element.

Compare:

- no positional encoding,
- RoPE,
- ALiBi,
- a simplified PaTH-style attention layer.

Measure accuracy as the number of swaps increases.



## Experiment 4: Householder Swap Verification

Construct

$$w = \frac{e_a - e_b}{\sqrt{2}}$$

and

$$H = I - 2ww^\top.$$

Verify numerically that

$$He_a = e_b, \quad He_b = e_a.$$

Then compose multiple Householder swaps and compare with explicit permutation matrices.

## Experiment 5: PaTH Key-Cache Update Benchmark

Implement the rank-one key-cache update

$$K \leftarrow K - \beta w(w^\top K)$$

and benchmark it for different:

- sequence lengths,
- head dimensions,
- numbers of heads,
- data types such as FP32, BF16, and FP16.

Compare the cost to ordinary attention decoding.

## Experiment 6: Inspect Learned PaTH Transformations

Train a small PaTH model on a state-tracking task and inspect:

- distribution of  $\beta_t$ ,
- norms of updated keys,
- whether some heads learn near-identity updates,
- whether some heads learn near-reflection updates,
- whether keys decay and become pruneable.

## Final Mental Model

RoPE turns position into geometry. Each token rotates query and key channels by an angle determined by its absolute index, but the dot product cancels the absolutes and leaves only relative displacement. High-frequency channels become a precise ruler for local order, while low-frequency channels carry semantic and long-range information. This is why RoPE is cheap, elegant, and widely used.

However, RoPE is still a system of fixed commutative rotations. It is good at measuring where tokens are relative to one another, but weak at representing ordered state updates. Many reasoning, coding, and symbolic tasks are not just about distance; they require applying a sequence of non-commutative transformations.

PaTH attention changes the role of positional encoding. Instead of asking, “How far apart are these tokens?”, it asks, “What transformations occurred between these tokens?” It uses cumulative products of data-dependent Householder-like matrices:

$$s_{ij}^{\text{PaTH}} = q_i^\top \left( \prod_{t=j+1}^i (I - \beta_t w_t w_t^\top) \right) k_j.$$

A Householder transform can be identity-like, projection-like, or reflection-like. Reflections can implement swaps. Compositions of swaps can implement permutation tracking. Therefore PaTH gives attention a mechanism for state evolution, not merely position measurement.

RoPE is a ruler. PaTH is a state-transition machine embedded inside attention. RoPE tells the model how far two tokens are apart; PaTH tells the model what happened between them.

### Visual Concept

Draw two parallel diagrams. On the left, show RoPE as independent circular rotations for channel pairs, with the attention score depending on the relative angle  $j - i$ . On the right, show PaTH as a chain of mirrors between token  $j$  and token  $i$ , where each mirror is a Householder transform and the final attention score depends on the composed transformation path.